OHM-ECDSA: Open Honest-Majority Threshold ECDSA from Public-Domain Components

Alp Guneysel*

Draft v0.8 — August 1, 2026

Abstract

We present OHM-ECDSA, a threshold signing protocol for ECDSA over secp256k1 (and generic prime-order groups) in the honest-majority setting ($n \geq 2T-1$, up to $T-1$ malicious parties), assembled exclusively from public-domain building blocks published between 1989 and 2007 — Shamir sharing, Feldman VSS, commit-then-reveal Pedersen DKG, Beaver triples, and Chaum–Pedersen DLEQ proofs — together with openly published presignature algebra (Groth–Shoup 2021, Katz 2023). The online signing phase is a single broadcast round. Every broadcast value is verified against public Feldman commitments by pure point equality, giving identifiable abort at every step with *unconditional* detection — no zero-knowledge range proofs, no homomorphic encryption, no oblivious transfer, no class groups; no NIZK proofs are needed for any share opening (the only proofs in the protocol are classical Chaum–Pedersen DLEQ proofs in triple generation). The construction is designed to practice no element of US Patent 11,757,657 (Sepior/Blockdaemon, covering DJNPO20) and to differ materially from the KU24 key-independent presignature pipeline (Dfns); we give an element-by-element claim chart and publish this work as defensive prior art. We provide a complete reference implementation (Rust), a companion networking crate running the full protocol between OS processes over TCP with signed messages and echo broadcast, and measurements: one-round online signing at 0.28 ms local compute (0.7 ms over a real mesh, RTT-bound at $\sim$2 link delays under WAN simulation), offline presignature generation at 5–10 ms per unit, and 1.4–1.8$\times$ throughput gains from packed (Franklin–Yung) sharing. Unlike the encumbered protocols, OHM-ECDSA additionally detects every wrong share unconditionally, attributes it cryptographically, and optionally guarantees output delivery.

Keywords. threshold signatures, ECDSA, honest majority, identifiable abort, Beaver triples, Feldman VSS, patent design-around.

1 Introduction

1.1 The patent situation

Threshold ECDSA deployments fall into two regimes. The dishonest-majority regime (GG18/GG20, DKLs, CGGMP) tolerates $T-1$ corruptions out of T signers at the cost of heavy machinery — Paillier-based multiplicative-to-additive conversion, range proofs, or OT — much of it patent-encumbered. The honest-majority regime ($n \geq 2T-1$), the natural model for key-management

*GitHub: `ungaro`. Project page, reference implementation, and full specification: `https://github.com/ungaro/ohm-ecdsa`.

networks where a fixed vetted committee shares keys on behalf of clients, replaces that machinery with information-theoretic arithmetic MPC; the concept is thirty-year-old public art (Gennaro–Jarecki–Krawczyk–Rabin, 1996).

Two modern refinements made honest-majority threshold ECDSA practical — and both are encumbered. DJNPO20 ("Fast Threshold ECDSA with Honest Majority") "appears covered by US Patent 11,757,657 assigned to Sepior ApS" (Katz, NIST MPTS 2023); Sepior has since been renamed Blockdaemon. KU24 ("Honest-Majority Threshold ECDSA with Batch Generation of Key-Independent Presignatures") is described by its authors' employer Dfns as "this patented protocol."

There is an irony here that motivates this work. ECDSA itself exists because Schnorr signatures were patented (US 4,995,082): NIST standardized DSA/ECDSA as the unencumbered workaround, and the workaround outlived the patent (expired 2008) to become the world's signature scheme. A signature scheme born as a patent design-around now has its threshold evolution patented — including claim elements built from the 1989–1996 public literature that the patent's own background cites.

1.2 Contributions

1. **An open construction (Section 3).** A complete honest-majority threshold ECDSA protocol — key generation, triple factory, presignature factory, one-round online signing — using only public-domain components, engineered so that the claim elements of US 11,757,657 simply never occur: the inverse $[k^{-1}]$ is *dealt directly* as a random sharing rather than computed by the patented masked-product inversion $[a] \cdot w^{-1}$; all products use Beaver triples with verified masked openings rather than the claimed zero-sharing blinding; and correctness is ensured by point equality against public Feldman commitments rather than the claimed honest-share-counting procedure.

2. **Unconditional detection with cryptographic attribution (Section 4).** Because every check is a point equality against a public commitment — which has exactly one passing scalar per position — every deviation is detected with probability 1 (unconditionally), and attributed to its sender (computationally, via ROM hash binding and message signatures). Identifiable abort itself is not new — GG20 and CGGMP21 provide it in the dishonest-majority regime at the cost of that regime's machinery — but neither DJNPO20 nor KU24 identifies cheaters, and what is new here is *unconditional detection from public-domain components in the honest-majority setting*. OHM-ECDSA additionally offers optional robustness (guaranteed output delivery) at no extra offline cost.

3. **A security statement and proof (Section 5).** We state the intended theorem and prove it game-by-game in the AGM+ROM under ECDLP (plus the Groth–Shoup presignature-ECDSA assumption): extraction for the DKG without rewinding, a bias-bounded uniformity lemma powered by identifiable abort itself, Feldman hiding in the AGM, and a composition lemma. The full write-up lives in `docs/proof/PROOF.md`; open items are named precisely (re-randomization lemma, UC, adaptive corruptions).

4. **A patent design-around analysis (Section 6).** An element-by-element claim chart against the verbatim granted text of US 11,757,657 claim 1, a distinction from KU24, and a prior-art wall. This document is itself a defensive publication.

5. **A complete implementation and evaluation (Section 7).** A dependency-pure Rust reference implementation of every protocol feature, plus a companion networking crate executing

the full protocol between separate OS processes over TCP with per-message signatures and echo broadcast — with blame tokens verifiable offline by a third party.

1.3 Related work

GJKR96 established robust threshold DSS with identifiable abort in the honest-majority setting; GJKR07 fixed the rushing attack on Pedersen DKG via commit-then-reveal. Beaver (1991) gave multiplication triples; Chaum–Pedersen (1992) the DLEQ proof; Franklin–Yung (1992) packed sharing; Bar-Ilan–Beaver (1989) constant-round arithmetic MPC with inverses. DJNPO20 and KU24 are the encumbered modern refinements discussed above. Groth–Shoup (2021) formalized ECDSA with presignatures and additive key derivation — our presignature layout and HD-tweak compatibility follow that model. Katz (NIST MPTS 2023) sketched the direct generation of $[k^{-1}]$ as a design-around for US 11,757,657; we adopt and complete that approach. cait-sith (cronokirby/NEAR, MIT) is an independent sibling implementation whose triple preprocessing is key-independent (its presignatures stay key-dependent) and which explicitly disclaims identifiable aborts. NEAR's production `near/mpc` additionally contains an MIT implementation of DJNPO20 itself, used by Chain Signatures. Guaranteed output delivery (fairness) is impossible with a dishonest majority in the plain model (Cleve 1986) — which is precisely why our robust variant operates in the honest-majority setting.

2 Preliminaries

Setting. n servers $P_1, \ldots, P_n$, threshold T, $n \geq 2T-1$; a static malicious adversary controlling up to $T-1$ servers; authenticated point-to-point channels and signed-echo consistent broadcast (the sender sends its signed value to all, receivers echo the sender's signed message plus their own signature, and a value is accepted when it carries the sender's signature plus $T-1$ echoes from other distinct parties with no conflicting sender-signed value seen — a conflicting pair forces $\perp$ and blames the sender — giving consistency and validity under honest majority). Parties are evaluation points $1..n$; point 0 is reserved for the secret. $\mathbb{G}$ is a prime-order group (secp256k1) of order q with generator G; $\mathbb{F}_q$ is its scalar field.

Components (all 1989–2007). Shamir sharing $[v]$ (degree $T-1$; party j holds $p(j)$); Feldman commitments $A[v] = (a_0 G, \ldots, a_{T-1} G)$ with share verification $v_j \cdot G == \sum_\ell j^\ell \cdot A_\ell :=$ EvalCom$(A[v], j)$; commit-then-reveal dealing (anti-rushing); Beaver triples $([\alpha], [\beta], [\gamma])$ with $\gamma = \alpha\beta$, and the masked-opening multiplication: open $\delta = x - \alpha$, $\varepsilon = y - \beta$, then $[xy] = [\gamma] + \delta[\beta] + \varepsilon[\alpha] + \delta\varepsilon$; Chaum–Pedersen DLEQ proofs for product commitments. H hashes to a scalar (ROM); $F(R)$ is the ECDSA x-coordinate extraction. A sharing together with its commitment is written $[\![v]\!]$ ("openable").

Verified opening. Open$([\![v]\!], S)$: parties broadcast shares; each receiver checks point equality against $A[v]$ and interpolates from the first T valid shares; any failing sender is blamed. All openings in OHM-ECDSA go through Open — identifiable abort is structural, not an add-on.

Timing model (as in SPEC §2.2). Safety — key privacy and unforgeability — does not depend on timing. Liveness and accepted-set consistency assume partial synchrony: a value counts as accepted only when its round completes (the round wait), so consistency rests on conflicting

evidence propagating within that wait. When actual delays exceed the wait, honest parties' outcomes can diverge (value-vs-$\perp$; in the worst schedule, two accepted values), but every divergence is attributable afterward — the conflicting sender-signed values are offline-verifiable evidence — and nothing is revealed, since every share opening is commitment-checked before use.

Presignature storage (as in SPEC §2.4/§8.6 — MUSTs). Presignature records are **single-use** and their shares are **key-equivalent**: T shares of $[u]$ and $[z]$ from the *same* record yield $x = z \cdot u^{-1}$, so pool compromise is key compromise. Deployments MUST store presignature shares with key-share-grade protection, MUST consume ids atomically (a consumed id is never usable again, even across crashes), MUST securely erase records on consume, expiry, or committee change, and MUST never use a record across keys. (cait-sith documents the analogous caveat for its own records.) The reference store enforces atomic consume and duplicate-id rejection; secure erasure is compiler-fenced with HSM-grade custody left to deployments.

3 The Construction

3.1 Key generation (commit-reveal Pedersen DKG)

Three rounds. **R1:** each P_i commits $h_i = H(\mathrm{sid} \,\|\, i \,\|\, A_i)$ to its Feldman commitment vector. **R2:** each reveals A_i and delivers shares over the authenticated channel. **R3:** hash and share checks; failures trigger a public complaint round in which the dealer's defense share decides dealer-vs-accuser blame. Output: $x_j = \sum_i s_{i,j}$, $A[x] = \sum_i A_i$, $X = \sum_i A_{i,0}$; the secret x exists nowhere. Commit-reveal is what makes the joint key uniform against a rushing adversary — and nonce uniformity is security-critical for ECDSA: any bias in k across signatures is exploitable by lattice key-recovery attacks.

3.2 Triple factory

Jointly generate random $[\![\alpha]\!], [\![\beta]\!]$ (two ephemeral commit-reveal VSS instances), then compute $[\gamma] = [\alpha\beta]$ by local products plus *verifiable degree reduction*: each P_j re-shares its product $\alpha_j\beta_j$ with a fresh degree-$(T-1)$ polynomial g_j and broadcasts $\mathrm{FeldCommit}(g_j)$ together with one DLEQ proof that $g_j(0) = \alpha_j\beta_j$ against the committed shares; the result combines with Lagrange weights: $[\gamma] = \sum_j \lambda_j [g_j]$. Correctness follows since $n \geq 2T-1$ points determine the degree-$(2T-2)$ product polynomial at 0. Batching (one commit-reveal covering the whole batch, constant rounds in B) and packed generation (Franklin–Yung: B secrets in one degree-$(T+B-2)$ polynomial, one product proof per party, $O(n^2)$ traffic for B triples, requiring $n \geq 2T + 2B - 3$) are implemented variants.

3.3 Presignatures (key-dependent)

Per presignature: deal $[\![u]\!]$ and $[\![a]\!]$ jointly; u ***is*** k^{-1} **by definition** — the inverse is generated directly, never computed (the Katz design-around). Open the Beaver product $v = a \cdot u$; set $[\![k]\!] := v^{-1}[\![a]\!]$, so $k = u^{-1}$ consistently. Nonce point: each party broadcasts $R_j = k_j \cdot G$, checked by $R_j == \mathrm{EvalCom}(A[k], j)$ — point equality, no proofs; $R = \sum_j \lambda_j R_j$, $r = F(R)$. Bind to the key with a second triple: open $\delta' = u - \alpha'$, $\varepsilon' = x - \beta'$ (the key masked by fresh uniform β'), then $[\![z]\!] = [\![\gamma']\!] + \delta'[\![\beta']\!] + \varepsilon'[\![\alpha']\!] + \delta'\varepsilon' = [k^{-1}x]$. Store $(\mathrm{id}, r, R, [u], [z])$ — single-use and key-equivalent (T shares of one record reveal x); the reference store enforces atomic consume.

3.4 Signing (one round)

Each party computes $s_j = m \cdot u_j + r \cdot z_j$ locally and broadcasts it; every share is verified against $m \cdot A[u] + r \cdot A[z]$ by point equality (blame on mismatch); interpolate $s = \sum_{j \in S} \lambda_j^S s_j$ from the first T valid shares; ECDSA-verify as defense in depth; normalize low-s; consume the id. Online cost per party: two scalar multiplications and share verification — sub-millisecond. Correctness is immediate: $s = mu + rz = k^{-1}(m + rx)$.

3.5 Extensions (implemented)

- **Robustness (guaranteed output delivery).** Bad shares are filtered and blamed; the honest majority finishes from the publicly committed values — online (filter-and-interpolate), offline openings (same), and triples (public reconstruction of the cheater's committed re-sharing polynomial — exclusion alone is impossible since $n - f$ can fall below the $2T-1$ product points needed).

- **Expel-and-restart.** Dealing-phase faults: blamed parties are removed and the instance restarts over the survivors if $n' \geq 2T-1$ (never lowering T); the aborted presignature id is poisoned. At minimal committees ($n = 2T-1$) expulsion forces re-sharing, which the next item provides.

- **Committee maintenance.** Proactive refresh (zero-constant re-sharing; X unchanged; shares from different epochs do not combine) and committee change (re-sharing to a new committee with public old-share binding $C_j[0] == \mathrm{EvalCom}(A[x], j)$).

- **HD key derivation.** For public tweak τ: $z'_j = z_j + \tau \cdot u_j$, $X' = X + \tau \cdot G$ — local, non-interactive, per the Groth–Shoup model. One presignature pool serves an entire BIP32-style tree. *Security caveat and safe-parameters argument:* AKD + presignatures is exactly the GS21 cube-root regime, whose attack has one precondition: the adversary knows a presignature's R (hence $r = F(R)$) *before* the message and tweak are fixed. The configuration then splits cleanly: **(external requesters)** if pool nonces are disclosed only at signing time — the deployment rule in SPEC §9.4 — the preprocessing window is zero and no degradation occurs; **(committee insiders)** the $\leq T-1$ corrupted parties see pool nonces at generation, the window is real, and the honest security level for HD-tweak signing in that configuration is the GS21 generic bound (∼85 bits at secp256k1), not 128. *Structural note:* the standard additive mitigation ($k' = k + \delta$) is *foreclosed* here — the protocol holds $[u] = [k^{-1}]$, and $(k + \delta)^{-1}$ has no local formula in k^{-1} shares. The inverse-compatible multiplicative form ($k' = \gamma k$, $u' = \gamma^{-1} u$, $z' = \gamma^{-1} z$, $\gamma = H(\mathrm{sid} \,\|\, \mathrm{id} \,\|\, M \,\|\, \tau \,\|\, X)$) is fully local and patent-clean. Its security analysis (`docs/proof/PROOF.md` §8.2) shows the feared offline evaluability of γ is not the attack's enabler: the cube-root attack needs the *affine* structure of a **constant** r, and per-candidate $r' = F(\gamma \cdot R)$ destroys it. The single-query condition is an unstructured RO fixed point at $O(q)$; the multi-candidate collision in $\Phi(M, \tau) = h(M) + F(\gamma \cdot R) \cdot \tau$ is shown (via a two-block meet-in-the-middle analysis) to cost birthday $\Theta(\sqrt{q})$ with the algebraic handles τ, τ' accounted — setting $\tau' = 0$ or $\tau = \tau'$ only makes the search harder. The no-collision residual is closed by conditional reduction to Groth–Shoup's Theorem 6 via their entropy-preservation generalization (the γ-map's precomputability is exactly what cases (1)–(2) charge for; forgeries not using the map are informationally identical in both models), giving the total bound $O(q_H^2/q) + O(q_s \cdot q_H/q) + \mathrm{Adv}_{\text{GS21-Thm6}}$. Status: **proof sketch; all attack cases analyzed**, with one named heuristic (G1: F-uniformity, which the GS21 attack itself also makes) and one model note (the case-(3) plug-in is in the GGM while cases (1)–(2) are ROM — the

standard two-layer composition). The G1 heuristic is additionally pressure-tested empirically: a control-validated Sage probe (`analysis/g1_probe.sage`; results in `analysis/README.md`) finds no sub-birthday strategy against Φ under adversarial tweaks at small scale (fitted exponents 0.47–0.53 across eight strategies, exact-table collision statistics matching a random function at ratio 1.0000), while its Wagner 4-sum positive control on the affine constant-r condition recovers the GS21 cube root (exponent 0.31) — small-scale evidence, not a proof. $k' = H(M\|\tau\|X) \cdot k$ is an RFC-6979-style message-derived nonce, so the attack's pre-computation window ceases to exist in the model. This makes OHM-ECDSA the second honest-majority scheme with a GS21 mitigation (after Groth–Shoup's service) and the only one compatible with direct inverse-dealing.

- **Key-independent mode (optional).** Pool records omit $[z]$; the binding moves online (two rounds, one triple — never degree-$(2T-2)$ local products, which are the patented machinery). One pool serves any key; unbound records are not key-equivalent. Patent posture in Section 6.

4 Identifiable Abort

Every broadcast value is one of: a hash that must match a later reveal, a share that must satisfy point equality against a public commitment, a DLEQ proof that must verify, or the final signature that must ECDSA-verify. Two facts make attribution unconditional rather than cryptographic:

Proposition 1 (Unconditional detection)**.** *A Feldman commitment vector $A[v] = (A_0, \ldots, A_{T-1})$ fixes each coefficient's point, and scalars map injectively to points. Hence the check $v_j \cdot G ==$ EvalCom$(A[v], j)$ has* exactly one *passing scalar per position j: a wrong share fails with probability 1, with no computational assumption. (Contrast Pedersen-style two-generator commitments, where binding is computational; here only* hiding *is computational.)*

Proposition 2 (Binding attribution)**.** *Each dealer is tied to its commitment vector by the R1 hash commit (binding of H in the ROM) and each message by its sender's signature; the complaint defense is publicly verifiable point equality. Blame evidence (the offending signed message plus the public commitment, a "blame token") is therefore verifiable offline by any third party.*

All abort decisions depend only on public verification outcomes, and every opened value is one-time-padded by a fresh uniform mask — so aborts leak nothing about x, u, z, or future presignatures. Contrast US 11,757,657's correctness procedure, which checks consistency *among* shares and, in its own three-party example, "can not determine which of the parties is dishonest."

5 Security Statement and Proof Skeleton

Security Statement 1. In the random-oracle model, under ECDLP in $\mathbb{G}$, OHM-ECDSA securely realizes the threshold-ECDSA functionality $\mathcal{F}_{\text{TECDSA}}$ against a static malicious adversary corrupting at most $T-1$ of $n \geq 2T-1$ servers, with identifiable abort — conditioned on the presignature-consumption invariant (single-use, key-bound, securely erased).

Claims. C1 (correctness). Honest execution outputs (r, s) with $s = k^{-1}(H(M) + rx)$: the invariant chain $v = au \Rightarrow k = u^{-1}$, $z = ux$, $s = mu + rz$, each step linearity given verified openings. **C2 (privacy).** The adversary's view is simulatable from X, public presignature data, and legitimately issued signatures (simulator below). **C3 (soundness/identifiability).** Unconditional, by Proposition 1. **C4 (nonce uniformity).** Commit-reveal plus at least one honest

dealer makes every joint secret uniform even against rushing adversaries; inversion is a bijection on $\mathbb{F}_q^*$, so uniform $u \Rightarrow$ uniform $k = u^{-1}$; the $v = 0$ restart rejects a $1/q$ fraction and does not bias u. Adversary-induced disqualification yields at most a keep/drop choice over its own contribution — unpredictable, not magnitude-controllable — and since even sub-bit structured nonce bias is fatal to ECDSA (Breitner–Heninger 2019; LadderLeak 2020), the L1 extraction proof must carry this bound explicitly. **C5 (unforgeability).** A forgery with fewer than T signing queries for that message reduces to ECDSA unforgeability in the Groth–Shoup presignature model. **C6 (robustness, optional).** After blaming $f \leq T-1$ cheaters, $n - f \geq T$ honest parties finish from publicly committed values.

Proof structure (game-based; full write-up in `docs/proof/PROOF.md`). **G0 (real).** Forgery is the only live winning condition: soundness-break and framing are impossible outright by C3. **G1 (program the DKG).** The simulator programs the commit-reveal so the joint public key is the challenge X (*L1 closure sketch, SPEC §11.5*: in the game-based ROM no rewinding is needed — honest R1 hashes are broadcast with content deferred, adversarial reveal vectors are read from the RO query log at R2 — any passing reveal was necessarily queried — and the honest vector is then programmed to land on X via exponent-Lagrange with random adversary-facing shares; UC porting expected but not claimed). The bias half of L1 is closed structurally: rejection-sampling the joint secret costs the adversary one expulsion per attempt, capping rerolls at $f \leq T-1$ and the residual selection bias at $O(\log T)$ bits (SPEC §11.5 — identifiable abort closing the bias gap, not merely adding accountability). **G2 (program every joint VSS)** likewise; fresh sids keep instances independent. **G3 (simulate DLEQ proofs)** via RO programming, $O(q_H^2/q)$ total. **G4 (program the openings)** using:

Lemma 1 (Freedom counting, sketch)**.** *For each degree-$(T-1)$ sharing, the adversary's $T-1$ shares plus one prescribed value (an opening, or X at point 0) leave exactly one consistent honest contribution; commitment vectors are programmed in the exponent to match. Hence the simulation of every masked opening is exact (statistical), not computational.*

The only computational residue is Feldman *hiding* of never-opened values (*gap L2, pinned in SPEC §11.6*: recommended route AGM+ROM, where the hiding hop becomes a lemma — any algebraic distinguisher yields a discrete log — leaving ECDLP the sole assumption; plain-model alternative $(T-1)$-OMDL with known shares). **G5 (sign from the oracle).** Sign queries are answered via the Groth–Shoup presignature-model oracle; honest s_j are computed by Lagrange through $(0, s)$ and the adversary's shares, consistent with $A[s] = mA[u] + rA[z]$ by construction. The reduction must additionally show that the adversary's extra view beyond the oracle's interface — per-party R_j, commitment vectors, DLEQ transcripts, opened masks — is simulatable from public data; this is where the G4 lemma is spent. **Extraction.** A forgery in the final game is a model forgery:

$$\mathrm{Adv}_{\mathrm{OHM}}(\mathcal{A}) \;\leq\; \mathrm{Adv}_{\text{GS-EUF}}(\mathcal{F}) \;+\; O(q_H^2/q) \;+\; \varepsilon_{L1} \;+\; \varepsilon_{L2}.$$

With L1 addressed by the closure sketch (SPEC §11.5) and L2 pinned to an AGM+ROM hiding lemma (SPEC §11.6), the remaining work toward a full proof is writing those two lemmas out in full plus a rigorous composition statement.

6 Patent Design-Around

US 11,757,657 (Sepior → Blockdaemon). The granted claim 1 is a conjunctive combination (quoted verbatim and mapped element-by-element in the specification): $[w] = [a][k]$; $R = g^k$ ensured correct "from at least $t+1$ shares originating from honest parties"; authenticator $W = R^a$

with its own check; $g^w == W$; and signing via $[k^{-1}] = [a] \cdot w^{-1}$, $[xk^{-1}]$, and $[s] = mw^{-1}[a] + rw^{-1}[a][x] + [d]$ with $[d]$ a random sharing of zero. Element by element: OHM-ECDSA has **no $[w]$, no W, no authenticator**; deals $[u] = [k^{-1}]$ **directly** (no inversion protocol); computes all products with **Beaver triples** over degree-$(T-1)$ sharings (**no zero-sharings** anywhere); and ensures correctness by **point equality against public commitments** — a different mechanism with a stronger result (attribution vs. mere detection). Shared elements (the ECDSA final algebra, share-exponentiation for R, abort-and-restart, one-round reveal) are the patent's own admitted prior art or unclaimed generic steps. Under the doctrine of equivalents, the inversion and correctness functions are performed in substantially different ways, with (for correctness) a substantially different result. Dependent claims 2–10 fail for the same reasons or read on admitted prior art.

KU24 (Dfns). KU24's contribution — and patent — is *batch generation of key-independent presignatures* via a coordinator-assisted pipeline. OHM-ECDSA is key-dependent by design (the P4 phase binds $[z] = [k^{-1}x]$ at generation time), which is precisely what keeps it clear. The optional key-independent mode (Section 3.5) follows earlier public art — DJNPO20's presignature is key-independent (2020); cait-sith shipped key-independent triple preprocessing (2022; its presignatures remain key-dependent) — and implements no element of KU24's pipeline (no coordinator, no KU24 packing); it is opt-in and flagged for FTO review. Dfns has publicly stated intent to lift the KU24 patent via LF Decentralized Trust.

Prior-art wall. Bar-Ilan–Beaver 1989; Beaver 1991; Pedersen 1991; Franklin–Yung 1992; Chaum–Pedersen 1992; GJKR 1996; GJKR 2007; Groth–Shoup 2021; cait-sith 2022; Katz MPTS 2023. This paper, timestamped publicly, is itself prior art against later patents on the construction.

Remark. This section is engineering analysis, not legal advice. Obtain a freedom-to-operate opinion for commercial deployment.

7 Implementation and Evaluation

7.1 Implementation

Two Rust crates. The **core** (`ohm-ecdsa`, ∼5,400 LoC) is dependency-pure (`k256`, `sha2`, `thiserror`, `rand`, `zeroize`), MSRV 1.75, no unsafe, and implements every feature of Section 3: DKG with public complaint arbitration, triple factory (single/batch/packed), presignatures (key-dependent, batched, packed, key-independent pools), one-round signing, robustness, expel-and-restart, refresh/re-sharing, single-use stores, HD tweaks, signed envelopes with offline-verifiable blame tokens, and a canonical wire format. The **node crate** (`ohm-ecdsa-node`) runs the full protocol between separate OS processes — strict key separation by construction — over full-mesh TCP with per-message ECDSA-signed envelopes and echo broadcast, with durable crash-safe single-use stores, transcript archiving, an offline blame-token auditor, and optional mTLS with committee certificate pinning. The node crate is hardened well beyond a demo: wire decoders are fuzzed (28M+ executions, zero crashes); connections self-heal via journaled reconnection with idempotent re-delivery; shutdown is clean (joined threads, deadlines on all blocking IO); DoS guards bound frame sizes and rates per peer; multiple protocol sessions run concurrently, demultiplexed by session id, behind a background presignature factory with target-level pooling and expiry; committee setup is a distributed ceremony in which no process ever holds another party's secret; robust continuation and expel-and-restart are available over the wire as opt-in modes; and key material is mlock-pinned in memory and AEAD-encrypted at rest with a KMS-pluggable storage key (env/command-plugin

interface). Store rollback is detected and refused at startup (journal + transcript cross-check); an operator metrics dump and a soak-test mode (continuous factory, periodic signing, fault injection, kill/rejoin cycles) support pre-production validation. The test suite comprises 193 tests (unit, integration, process-level with real child processes, and example smoke tests), deterministic throughout.

7.2 Measurements

Single-threaded reference driver (Apple M4 Max, medians): KeyGen 0.97 ms; triple 3.2 ms; presign 9.7 ms; **online sign 0.28 ms** (2-of-3). Packed (Franklin–Yung) generation vs. per-item batching: **1.43**× (2-of-5, B=2) to **1.83**× (3-of-9, B=3) on triples, growing with n and B as the accounting predicts (dealing drops from $2B$ polynomials to 2; product proofs from $B{\cdot}n$ to n). Over the real TCP mesh (per-party processes): keygen 1.8 ms and sign **0.7 ms** on localhost; under a 50 ms/link WAN simulation, sign 113 ms ≈ 2 link delays and keygen 337 ms ≈ 6+ — confirming that online latency is round-trip-bound, not compute-bound, with one round online.

7.3 Comparison with DJNPO20 and KU24

Machine-independent rows (timings quoted from the papers under different hardware and network conditions — context, not a shootout):

	DJNPO20	KU24	OHM-ECDSA
Online rounds	1	non-interactive[†]	1
Identifiable abort	no	no	**yes**[*]
Guaranteed delivery	no[‡]	no	**yes (optional)**
Presignatures	key-independent	key-independent, batched	key-dependent[§]
Reported presig cost	34 ms (AWS LAN)[a]	1.3 ms amortized[¶]	9.7 ms; 5.2 ms packed
Reported online cost	19.9 ms e2e[a]	≈80 µs local + net[b]	0.28 ms local; 0.7 ms mesh
Coordinator	none	semi-honest, required	none
Assumptions	ECDLP	ECDLP + GGM + coord.	ECDLP + ROM
Patent status	US 11,757,657	patented per Dfns	none; prior-art published

Table 1: Feature and performance comparison. [†]plus a semi-honest coordinator. [‡]optional fairness extension. [§]with opt-in key-independent pools. [¶]at batch size 10,000 with a coordinator; 680 ms unbatched at 50 ms delay. [*]unconditional detection with computational attribution; identifiable abort exists in the dishonest-majority regime (GG20, CGGMP21) at the cost of that regime's machinery. [a]DJNPO20 Table 2, p. 13. [b]KU24 Table 1, p. 16.

KU24's 1.3 ms/presig is the amortization of key-independent batching at m = 10,000 behind a coordinator — a different feature set. DJNPO20's 19.9 ms "sign" includes client round-trip and database; the machine-independent comparison is one online round each. OHM-ECDSA's differentiators are accountability properties the encumbered protocols do not offer at any speed: unconditional detection of wrong shares with cryptographic attribution, and optional guaranteed delivery, with no coordinator.

8 Limitations and Future Work

Security status (updated). The game-based security claim is now **proved** in the AGM+ROM under ECDLP and the Groth–Shoup presignature-ECDSA assumption — see the working proof

document (`docs/proof/PROOF.md`, v0.2): C1 (correctness), C3 (blame soundness and framing-freeness), C4 (uniformity with the bounded-bias lemma), C6 (robustness) and the abort-distribution lemma, L1 (DKG extraction, ROM, no rewinding), L2 (Feldman hiding in the AGM — representation extractor, basis induction, and simultaneous embedding all written, no factor-Q union loss), and the composition lemma are proved there with per-hop bounds. Two honest notes: the model is *stronger* than the plain-ROM target (the AGM is needed for Feldman hiding and is the Groth–Shoup model anyway — ECDLP remains the sole hardness assumption), and the proof has not yet had external review. **Open theoretical items:** the re-randomization lemma (the candidate mitigation of Section 3.5 — proof sketch with one named heuristic, empirically probed as described there), UC and adaptive-corruption extensions (the claim is static-only). **Assurance evidence (implementation level):** the reference crates carry a consolidated F1–F8 fault-injection blame matrix, property-based tests over the algebraic invariants, byte-pinned deterministic test vectors (every sign vector verified with an independent k256 verifier), and a claims→code→test traceability map (`docs/TRACEABILITY.md`) with the reviewer-facing status table in `docs/PROOF_STATUS.md`.

Implementation status. The node crate is a hardened reference transport, not certified production infrastructure: store rollback is *detected* and refused (journal + transcript cross-check), but whole-directory rollback prevention needs external state (HSM counter or peer attestation); HSM/KMS integration is an interface (env/command-plugin), not an implementation; crash recovery of finished rounds and committee rejoin after a full restart remain; an external audit is still required before any production use. PRSS-based randomness was analyzed and deliberately deferred: PRSS outputs carry no Feldman commitments, conflicting with unconditional detection.

Errata (v0.5): the simple majority-echo acceptance rule of earlier drafts was found inconsistent at $T \geq 3$ (two size-T quorums at $n = 2T-1$ may intersect only in corrupt parties); the broadcast primitive has been strengthened to signed-echo consistent broadcast — acceptance requires the sender's signature plus $T-1$ signed echoes and the absence of any conflicting sender signature, with equivocation yielding $\perp$ plus an offline-verifiable blame token (new fault class F8).

References

[1] D. Beaver, *Efficient Multiparty Protocols Using Circuit Randomization*, CRYPTO 1991.

[2] J. Bar-Ilan, D. Beaver, *Non-Cryptographic Fault-Tolerant Computing in Constant Number of Rounds of Interaction*, PODC 1989.

[3] D. Chaum, T. P. Pedersen, *Wallet Databases with Observers*, CRYPTO 1992.

[4] I. Damgård, T. P. Jakobsen, J. B. Nielsen, J. I. Pagter, M. B. Østergaard, *Fast Threshold ECDSA with Honest Majority*, IACR ePrint 2020/501 (DJNPO20).

[5] M. Franklin, M. Yung, *Communication Complexity of Secure Computation*, STOC 1992.

[6] R. Cleve, *Limits on the Security of Coin Flips When Half the Processors Are Faulty*, STOC 1986.

[7] R. Gennaro, S. Jarecki, H. Krawczyk, T. Rabin, *Robust Threshold DSS Signatures*, EUROCRYPT 1996 (GJKR96).

[8] R. Gennaro, S. Jarecki, H. Krawczyk, T. Rabin, *Secure Distributed Key Generation for Discrete-Log Based Cryptosystems*, J. Cryptology 20(1), 2007 (GJKR07).

[9] J. Groth, V. Shoup, *On the security of ECDSA with additive key derivation and presignatures*, IACR ePrint 2021/1330.

[10] T. P. Jakobsen, I. B. Damgård, M. B. Østergaard, J. B. Nielsen, *Method for providing a digital signature to a message*, US Patent 11,757,657 B2 (granted 2023; Sepior ApS → Blockdaemon ApS).

[11] J. Katz, *Threshold ECDSA: advances and open problems*, NIST MPTS 2023.

[12] J. Katz, A. Urban, *Honest-Majority Threshold ECDSA with Batch Generation of Key-Independent Presignatures*, IACR ePrint 2024/2011 (KU24).

[13] L. Meier (cronokirby), *cait-sith* — threshold ECDSA via committed Beaver triples (MIT), 2022–2023. `https://github.com/cronokirby/cait-sith`; NEAR production derivative: `https://github.com/near/mpc`.

[14] T. P. Pedersen, *Non-Interactive and Information-Theoretic Secure Verifiable Secret Sharing*, CRYPTO 1991.

[15] C. P. Schnorr, *Method for identifying subscribers and for generating and verifying electronic signatures in a data exchange system*, US Patent 4,995,082 (1991; expired 2008).

[16] J. Breitner, N. Heninger, *Biased Nonce Sense: Lattice Attacks against Weak ECDSA Signatures in Cryptocurrencies*, FC 2019.

[17] D. F. Aranha, F. R. Novaes, A. Takahashi, M. Tibouchi, Y. Yarom, *LadderLeak: Breaking ECDSA with Less than One Bit of Nonce Leakage*, CCS 2020.

[18] J. Groth, V. Shoup, *Design and analysis of a distributed ECDSA signing service*, IACR ePrint 2022/506.